Disclaimer: This is a fictional project developed to demonstrate my MuleSoft skills.

Ecoform Store

API-led Architecture MuleSoft / CloudHub 2.0

Mahassen ESSALHI

Technical Presentation · 50 min

Ethical Fashion Store • 5 MuleSoft APIs • Neon PostgreSQL • Salesforce CRM • OpenWeatherMap

Agenda — 50 minutes

01 **Specs — Context & Objectives** — 5 min
Business problem, API-led architecture, sequence diagrams

02 **RAML — API Design** — 5 min
Starter project, global library, data models, API specifications

03 **Anypoint Studio** — 15 min
Logs, structure, properties, error handling, DataWeave, unit tests, POM

04 **Business Flows (01, 02, 03)** — 15 min
Catalog, Salesforce Scheduler, Order Status & Weather

05 **Evolutions** — 5 min
future features

06 **Git, GitHub** — 5 min
code versioning

01

Specs — Context & Objectives

Ecoform Store — Online Ethical Fashion · Problem & Architecture

Project Context

Ecoform Store

Online ethical fashion store — organic clothing, linen, organic cotton. Full e-commerce platform with catalog, orders and delivery tracking.

MuleSoft Platform

Anypoint Studio + CloudHub 2.0 — API-led architecture in 3 layers (Experience, Process, System). Deployed on us-east-1.

Database

Neon Tech (PostgreSQL serverless) — Product catalog, orders, customers and addresses.

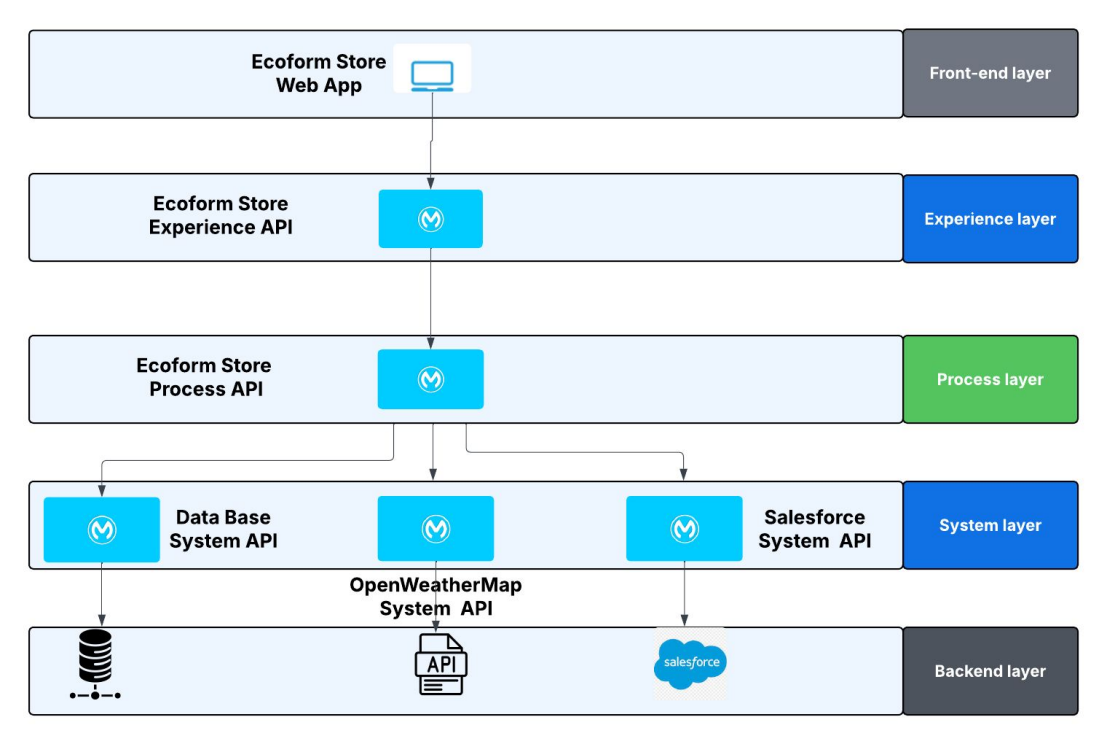
Salesforce CRM

Sales Cloud — batch synchronization of orders and customers every 60 min.

Weather API

OpenWeatherMap — dynamic delivery delay calculation based on weather conditions.

API-led Architecture — Overview



The 5 APIs — Responsibilities

exp-storefront

8084 **Experience**

Consumer entry point
GET /products · GET /order-status/{orderId}

prc-storefront

8084 **Process**

Orchestration & business logic
GET /products · GET /order-status · Batch CRON

sys-database

8083 **System**

Neon PostgreSQL (JDBC)
GET /products · PATCH /orders · GET /orders/{id}

sys-salesforce

8081 **System**

Salesforce CRM (Bulk API v2)
POST /accounts · /orders · /order-items

sys-weather

8082 **System**

OpenWeatherMap (REST)
GET /weather?zipCode=

Database — Neon Tech (PostgreSQL)

mulesoft

Tables

production

neondb

Database studio

public

Search...

- address
- customer
- order
  - order_id ASC
- order_item
- product

Filters | Columns | + Add record

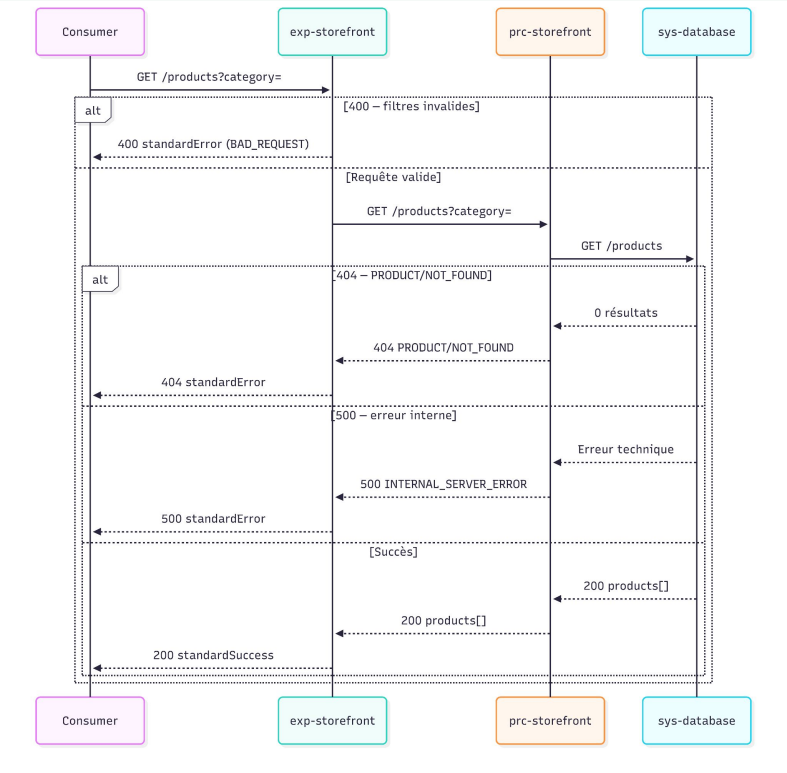
order_id varchar(20)	customer_id varchar(10)	shipping_address_id uuid
ORD-2026-00001	CUST-0001	a1000000-0000-0000-000…
ORD-2026-00002	CUST-0002	a1000000-0000-0000-000…
ORD-2026-00003	CUST-0003	a1000000-0000-0000-000…
ORD-2026-00004	CUST-0004	a1000000-0000-0000-000…
ORD-2026-00005	CUST-0005	a1000000-0000-0000-000…
ORD-2026-00006	CUST-0006	a1000000-0000-0000-000…
ORD-2026-00007	CUST-0007	a1000000-0000-0000-000…
ORD-2026-00008	CUST-0008	a1000000-0000-0000-000…
ORD-2026-00009	CUST-0009	a1000000-0000-0000-000…
ORD-2026-00010	CUST-0010	a1000000-0000-0000-000…

02

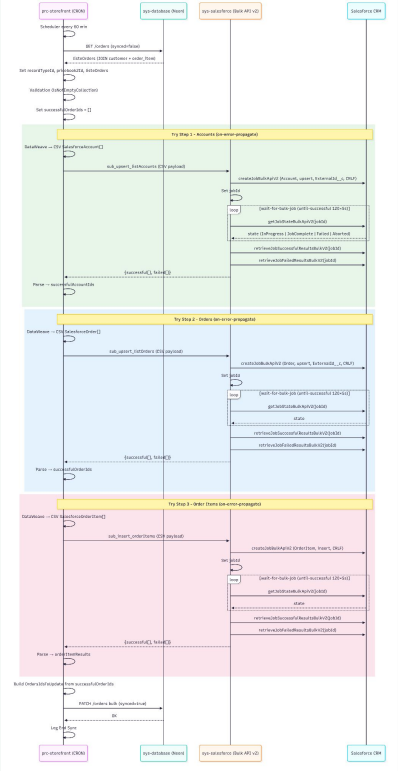
Sequence Diagrams

Communication flows between API-led layers

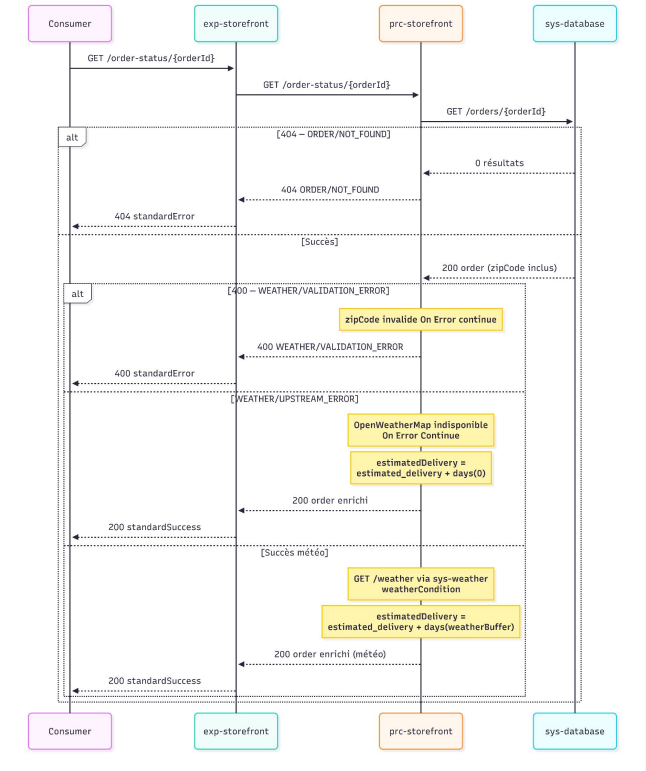
Sequence — Flow-01: Product Catalog



Sequence — Flow-02 : Salesforce Synchronization



Sequence — Flow-03: Order Status



03

RAML — API Design

Starter project · Global library · Data models · API specifications

Starter Project & Global Library

Starter Project

API creation template integrating a global library. Standardized structure for all project APIs, with pre-configured error handling and unified responses.

Global Library (Exchange)

Library published on Anypoint Exchange. Unifies responses (standardSuccess) and error handling (error400, error404, error500) across all APIs.

Project Data Types

Product ,Order , Address,Customer,OrderItem
Types shared via Exchange modules with validation patterns (e.g.: ^[A-Z]+-ECO-\d{3}$, ^ORD-\d{4}-\d{5}$)

DataTypes RAML

order	API fragment	March 30th, 2026	Mahassen
Starter Project API	API specification	March 27th, 2026	Mahassen
customer	API fragment	March 25th, 2026	Mahassen
globalLibrary	API fragment	March 25th, 2026	Mahassen
orderItem	API fragment	March 25th, 2026	Mahassen
address	API fragment	March 25th, 2026	Mahassen
product	API fragment	March 25th, 2026	Mahassen

exp-storefront-api/master

Share

```
#%RAML 1.0
title: exp-storefront
version: v1

securedBy: [lib.clientIdEnforcement]

uses:
  lib: /exchange_modules/28751335-e36c-47d2-abe4-ade1288e1a93/globallibrary/1.0.1/global-library.raml

types:
  Product: !include exchange_modules/28751335-e36c-47d2-abe4-ade1288e1a93/product/1.0.0/product.raml
  Order:   !include exchange_modules/28751335-e36c-47d2-abe4-ade1288e1a93/order/1.0.0/order.raml

# ────────────────────────────────────────────────────────────────────────
# /products — Flux-01 GET catalogue
# ────────────────────────────────────────────────────────────────────────
/products:
  securedBy: [lib.clientIdEnforcement]
  description: |
    Consultation catalogue produits — Flux-01.
    Sécurisé par Client ID Enforcement (headers client_id + client_secret).
    Délègue à prc-storefront → sys-database.

  get:
    is: [lib.error400, lib.error404, lib.error500]
    queryParameters:
      category:
        type: string
        required: false
        enum: [Haut, Bas, Robe, Manteau, Accessoire]
        description: Filtre sur la catégorie du produit
        example: Haut
    responses:
      200:
        body:
          application/json:
            type: lib.standardSuccess
```

Documentatio

API title: exp-storefro

Version: v1

API endpoints

/products

GET

/order-status/{orderId}

GET

04

Anypoint Studio

Logs · Structure · Properties · Error Handling · DataWeave · Tests · POM

Logging Strategy

```
<!-- Mule logger -->
<AsyncLogger name="prc-storefront" level="INFO"/>
<AsyncLogger name="prc-storefront" level="DEBUG"/>
```

Log Structure

Structured logs with correlationId for inter-API traceability. Each flow logs: start/end of processing, key business steps, errors with normalized namespace and identifier.

Log Placement

- Entry/Exit of each main flow
- Before and after each system call (DB, SF, Weather)
- In on-error blocks: error logged + propagated
- Batch results: successful/failed per step

Logging Strategy

```
1  %dw 2.0
2  output application/json
3  ---
4  {
5      step: vars.metadata.startStep,
6      data: {
7          business:  {successful: sizeOf(vars.accountResults.successful),
8          failed: sizeOf(vars.accountResults.failed)},
9          technical: {
10             flowId: vars.metadata.flowId,
11             correlationId: vars.metadata.correlationId
12         }
13     }
14 }
```

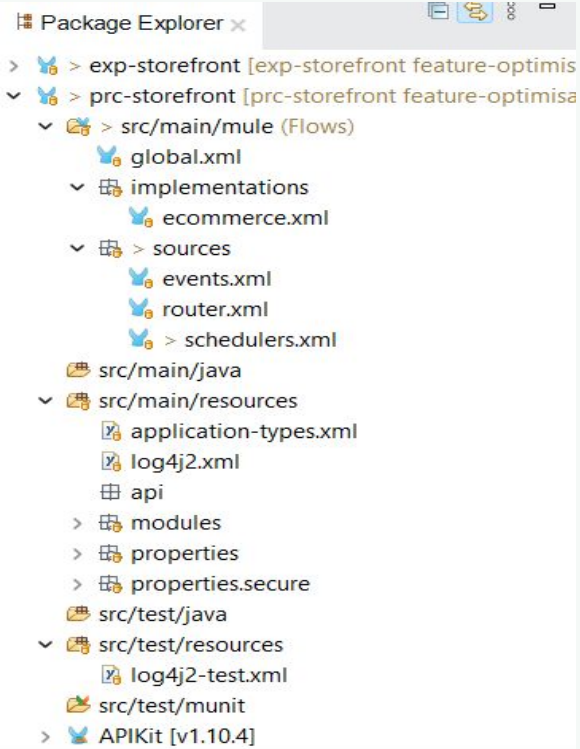
Anypoint Studio — Structure & Starter Project

Folder Structure

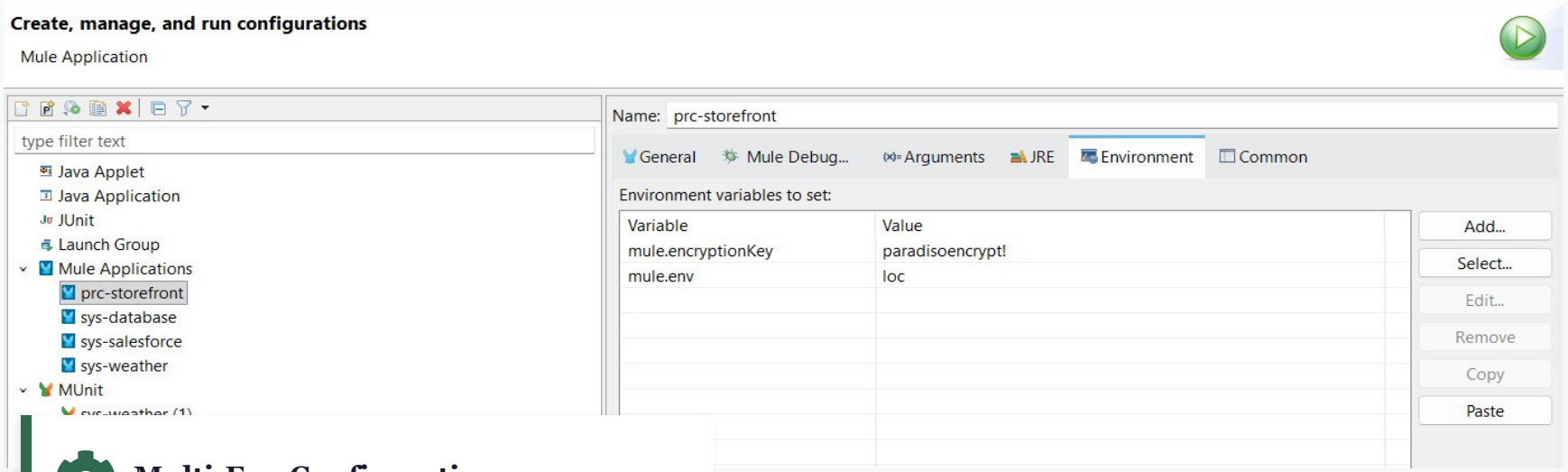
Standardized layout: src/main/mule (flows), src/main/resources (properties, configs), src/test (MUnit). The starter project provides a pre-configured template with dependencies and the global library.

Benefits of the Starter Project

Standardizes the structure across all 5 APIs. Automatically integrates: global library, global error handler, secure properties, multi-environment config (dev/prod). Saves time at initialization.



Environment Properties



Multi-Env Configuration

Properties files separated by environment (dev/prod). Externalized hosts, ports, credentials. Secure properties for secrets.

prod-properties.yaml

```yaml
################################
# API Manager
autodiscoveryId: "20840805"

# Listener
https:
  port: "443"
################################
sys-database:
  port: "443"
  host: "sys-database-4ex0or.5sc6y6-4.usa-e2.cloudhub.io"
  basePath: "/api/v1"
  protocol: "HTTPS"
  timeout: "300000"
sys-weather:
  port: "443"
  host: "sys-weather-4ex0or.5sc6y6-4.usa-e2.cloudhub.io"
  basePath: "/api/v1"
  protocol: "HTTPS"
  timeout: "300000"
sys-salesforce:
  port: "443"
  host: "sys-salesforce-4ex0or.5sc6y6-3.usa-e2.cloudhub.io"
  basePath: "/api/v1"
  protocol: "HTTPS"
  timeout: "300000"
  RecordTypeId: "012g5000001eWqDAAU"
  Pricebook2Id: "01sg5000001cReYAAU"


salesforce:
  tokenUrl: "https://orgfarm-7698c51d75-dev-ed.develop.my.salesforce.com/services/oauth2/token"
  clientId: "${secure::salesforce.clientId}"
  clientSecret: "${secure::salesforce.clientSecret}"
```

Security & Property Encryption

```yaml
################################
# API Manager
autodiscoveryId: "20840206"

# Listener
https:
  port: "8081"
################################

salesforce:
  tokenUrl: "https://orgfarm-7698c51d75-dev-ed.develop.my.salesforce.com/services/oauth2/token"
  clientId: "${secure::salesforce.clientId}"
  clientSecret: "${secure::salesforce.clientSecret}"
```

Secure Properties: ${secure::db.password}, ${secure::oauth.clientSecret}, ${secure::weather.apiKey} — encrypted values in configuration files.

Configuration & Secure Connectivity

Secure Properties

- ${secure::db.password} — encrypted Neon password
- ${secure::oauth.clientSecret} — SF client secret
- ${secure::weather.apiKey} — OpenWeatherMap API key
- .yaml files encrypted per environment

Salesforce OAuth

- Connected App — Client Credentials Grant
- Token URL: login.salesforce.com/.../token
- Scope: api refresh_token
- Automatic AccessToken refresh

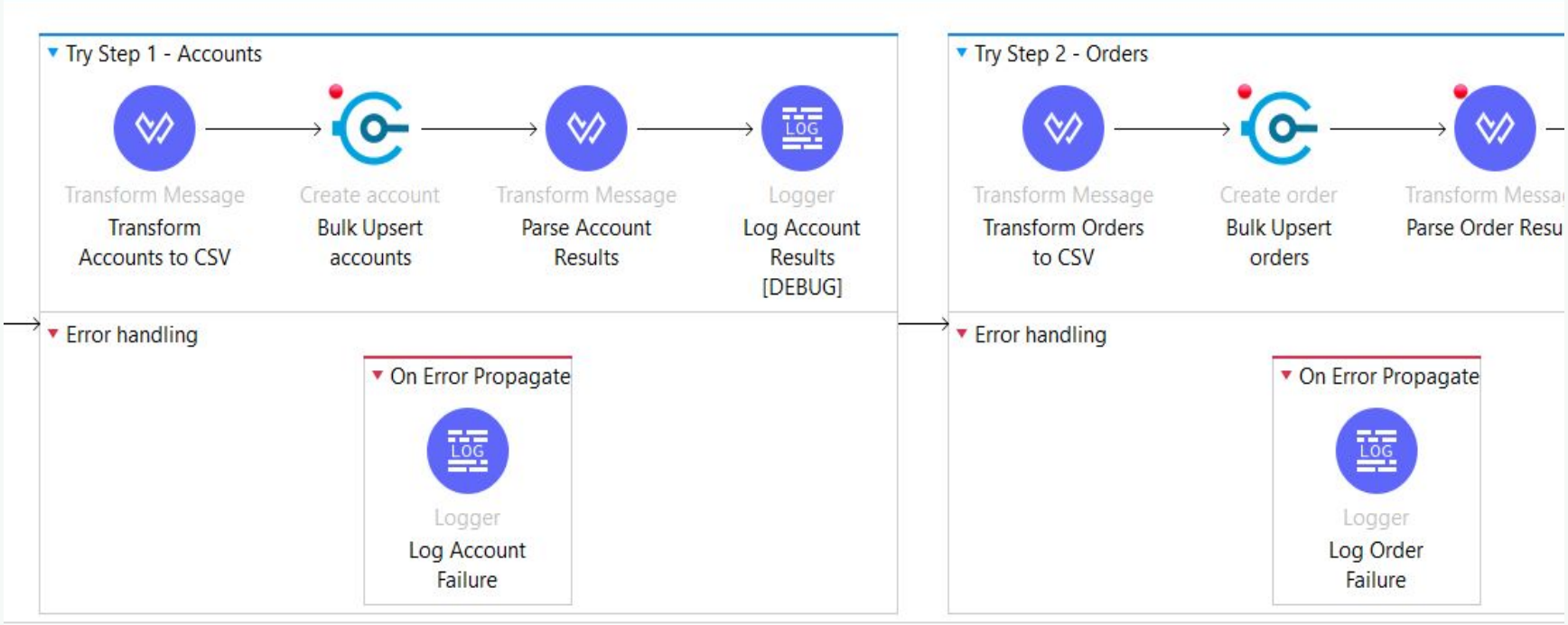
Property Variables

- sys-database.host / port / basePath / protocol
- sys-salesforce.host / timeout / protocol
- Pricebook2Id, RecordTypeId hardcoded
- Full externalization of configs

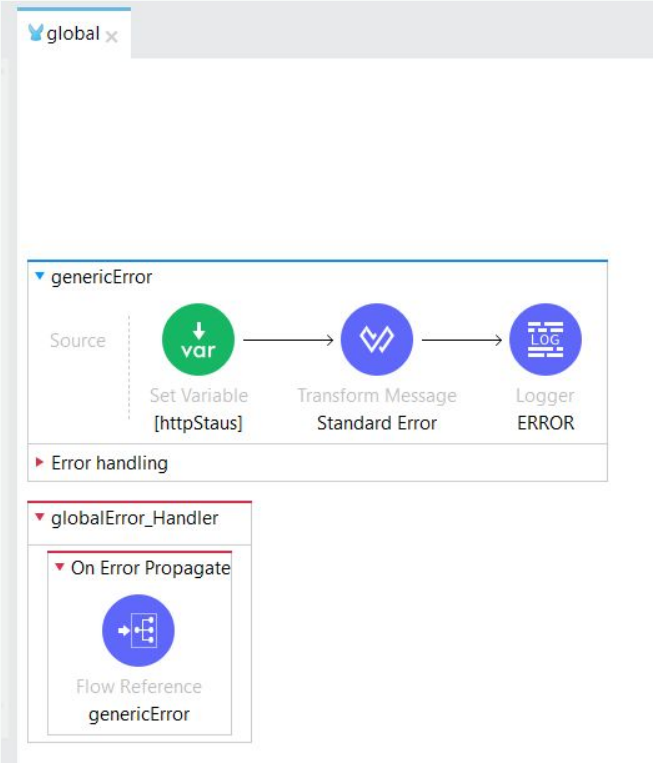
CloudHub 2.0

- Deployment Region: us-east-1
- Worker Size: 0.1 vCores / 0.5 GB RAM
- Mule Runtime 4.11.0 EE
- Integrated Monitoring & alerting

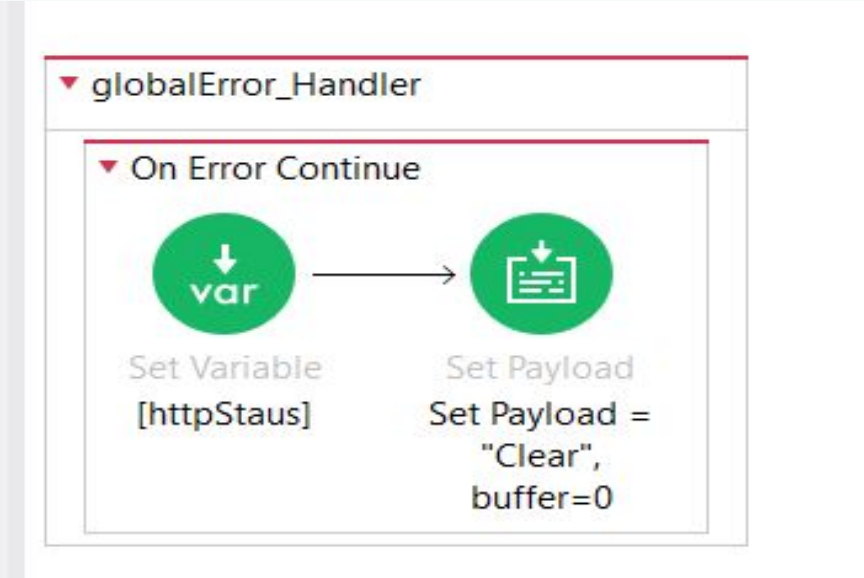
Error Handling — Try / Catch Block



Error Handling — Global Error Handler



Weather Resilience — On Error Continue



Fallback Strategy

In case of error on sys-weather:

- On Error Continue (no propagation)
- weatherBuffer = 0
- shippingNotice = "Standard delivery"
- Service continuity guaranteed

The client always receives a valid response, even if the weather API is unavailable.

05

DataWeave

Transformation algorithms

DataWeave — Order Mapping with weather-based delivery date calculation

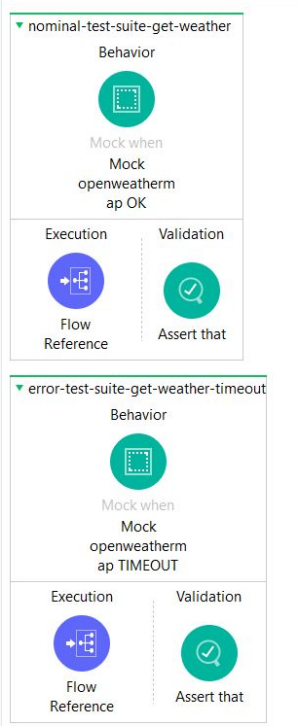
```
---

do {
    var order             = vars.order
    var weatherCondition  = (vars.weatherCondition default "Clear") as String
    var standardDelay     = order.standard_delay default 5
    var weatherBuffer     = getWeatherBuffer(weatherCondition)
    var baseDelivery      = order.estimated_delivery as Date {format: "yyyy-MM-dd"}
    var estimatedDelivery = (baseDelivery + days(weatherBuffer)) as String {format: "yyyy-MM-dd"}
    ---
    {
        orderId           : order.order_id,
        status            : order.status,
        paymentStatus     : order.payment_status,
        paymentMethod     : order.payment_method,
        synced            : order.synced,
        shippingAddress   : {
            zipCode : order.shipping_zip_code,
            country : order.shipping_country
        },
        orderTotal : {
            amount   : order.order_total_amount,
            currency : order.order_total_currency
        },
        weatherCondition  : weatherCondition,
        standardDelay     : standardDelay,
        weatherBuffer     : weatherBuffer,
        estimatedDelivery : estimatedDelivery,
        shippingNotice    : getShippingNotice(weatherCondition)
    }
}
```

06

Unit Tests

MUnit — Automated flow validation

Unit Test — MUnit



Testing Approach

MUnit tests to validate critical flows:

- Mock of external connectors (Weather)
- Error scenario tests (fallback)

Parent POM — Dependency Management

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w
3      <modelVersion>4.0.0</modelVersion>
4      <parent>
5          <groupId>[redacted]</groupId>
6          <artifactId>paradiso-parent-pom</artifactId>
7          <version>1.0.0</version>
8      </parent>
```

Centralization with Parent POM

A parent POM centralizes dependency versions (Mule Runtime, connectors, plugins) for all 5 APIs. Ensures version consistency and simplifies cross-cutting updates. Each API inherits from the parent without duplicating Maven configuration.

07

Flow–01 — 02—03

API List

API	Méthode	Endpoint
System Layer — sys-database (sécurisé : Client ID Enforcement)		
sys-database	GET	/products
	GET	/orders
	GET	/orders/{orderId}
	PATCH	/orders
System Layer — sys-weather (OpenWeatherMap) (sécurisé : Client ID Enforcement)		
sys-weather	GET	/weather
System Layer — sys-salesforce (Salesforce CRM — Bulk API v2)		
sys-salesforce	POST	/accounts
	POST	/orders
	POST	/order-items
Process Layer — prc-storefront (sécurisé : Client ID Enforcement)		
prc-storefront	GET	/products
	GET	/order-status/{orderId}
	POST	/batch/salesforce-sync
Experience Layer — exp-storefront (sécurisé : Client ID Enforcement)		
exp-storefront	GET	/products
	GET	/order-status/{orderId}

08

Flow-02 — Synchronization Salesforce CRM

CRON 60 min · Bulk API v2 · DataWeave CSV · Upsert Account/ UpsertOrder/Insert OrderItem

Salesforce Data Model (Flow-02)

Account

Person Account
ExternalId__c = customerId
RecordTypeId required

Pricebook2

Standard Price Book

Order

Ecoform Order
ExternalId__c = orderId
Pricebook2Id required

PBEntry

PricebookEntry
Product2 + Pricebook2 Link
ExternalId__c = productId
Reference for OrderItem

OrderItem

Order line (insert)
PricebookEntryId required
TotalPrice auto-calculated by SF

SF org prerequisites: Multi-Currency enabled + EUR | PersonAccount enabled | ExternalId__c created on Account/Order/PricebookEntry | Custom fields on Order and OrderItem

Flow-02 — Scheduler Architecture (8 steps)

1. **GET /orders (synced=false)**
   `sys-database`

2. **DataWeave → CSV Account[]**
   `prc-storefront`

3. **Upsert Accounts (Bulk v2)**
   `sys-salesforce`

4. **DataWeave → CSV Order[]**
   `prc-storefront`

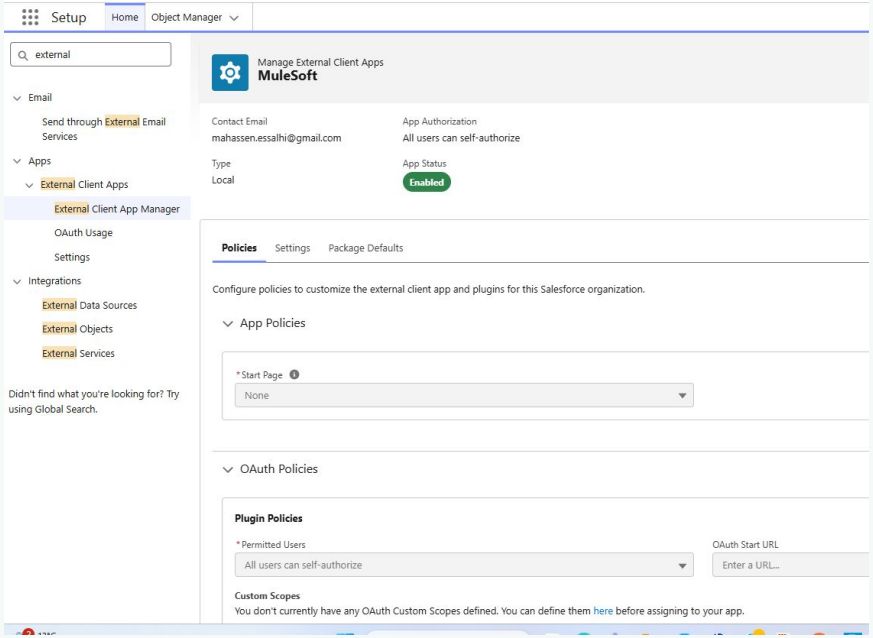
5. **Upsert Orders (Bulk v2)**
   `sys-salesforce`

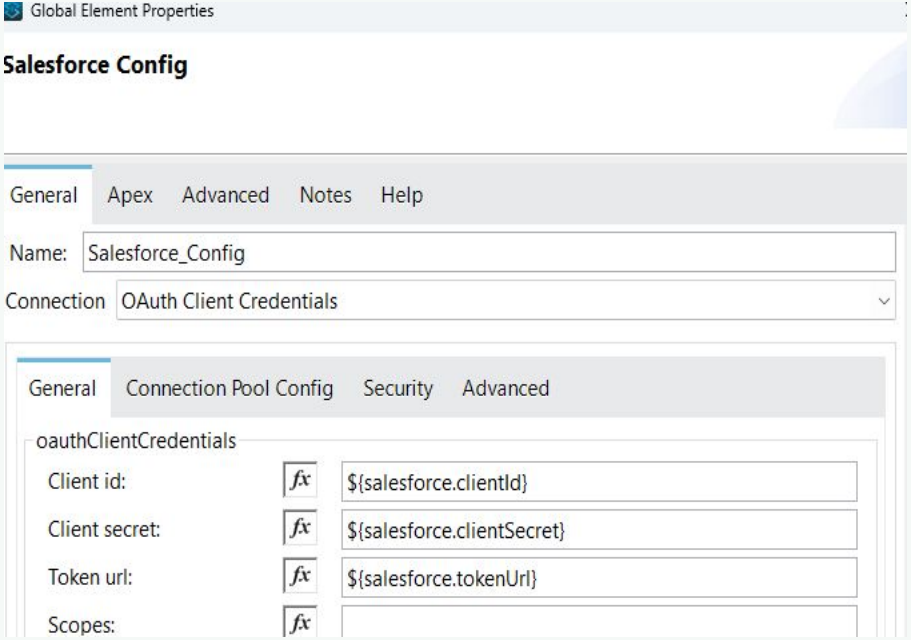
6. **DataWeave → CSV OrderItem[]**
   `prc-storefront`

7. **Create OrderItems (Bulk v2)**
   `sys-salesforce`

8. **PATCH /orders synced=true**
   `sys-database`

Salesforce — Connected App & MuleSoft Configuration





OAuth Client Credentials — Native MuleSoft Salesforce Connector — Bulk API v2 (CSV, lineEnding=CRLF)

Linking Salesforce Objects via ExternalId__c

```
1 %dw 2.0
2 output application/java
3 ---
4 vars.listeOrders map (ord) -> {
5     ExternalId__c        : ord.order_id,
6     Account              : { ExternalId__c: ord.customer_id, "type": "Account" },
7     Pricebook2Id         : vars.pricebook2Id ,
8     Status               : ord.status
```

Relation	DataWeave Syntax	SF Resolution
Order → Account	`{ ExternalId__c: customerId, "type": "Account" }`	SF resolves AccountId automatically
OrderItem → Order	`{ ExternalId__c: orderId, "type": "Order" }`	SF resolves OrderId automatically
OrderItem → PricebookEntry	`{ ExternalId__c: productId }`	SF resolves PricebookEntryId automatically

Salesforce — API Call Verification

SETUP

Bulk Data Load Jobs

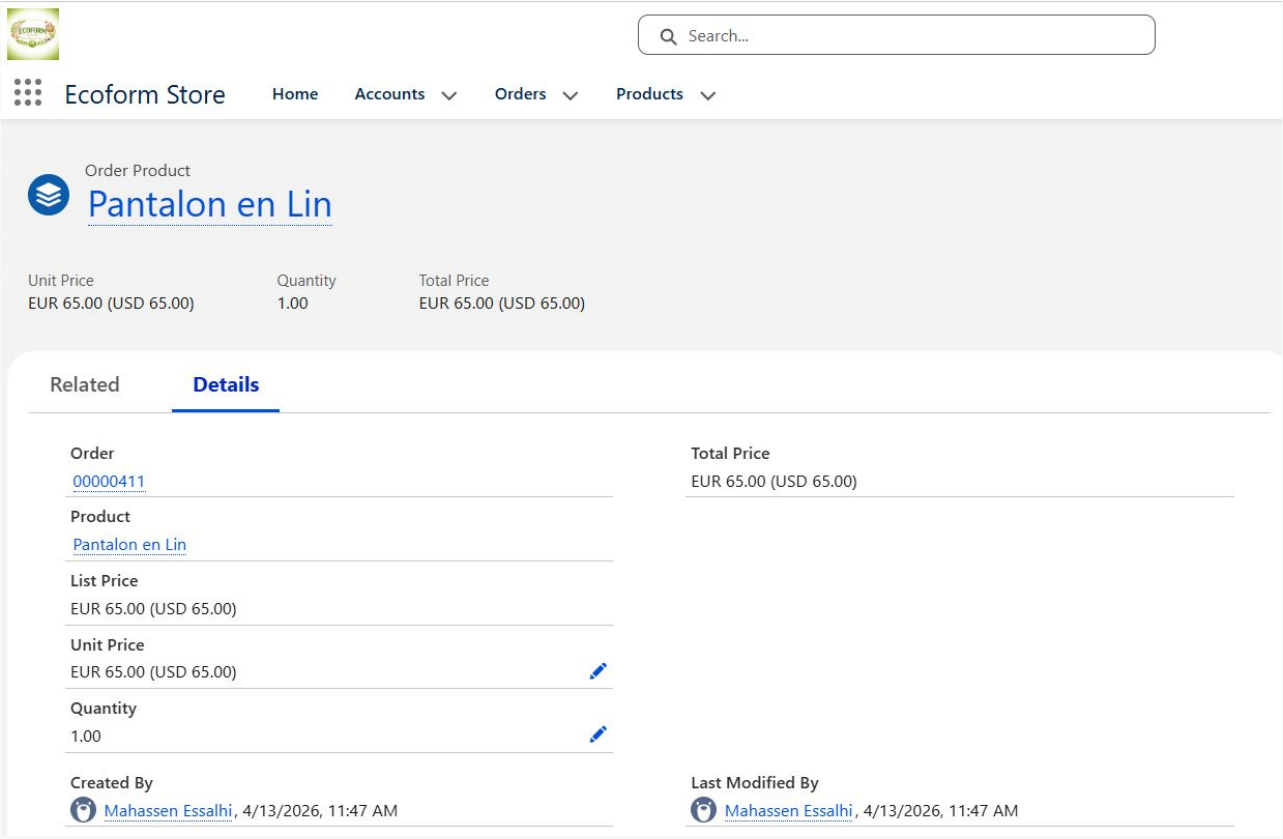
« Back to List: Bulk Data Load Jobs

Bulk Data Load Job Detail

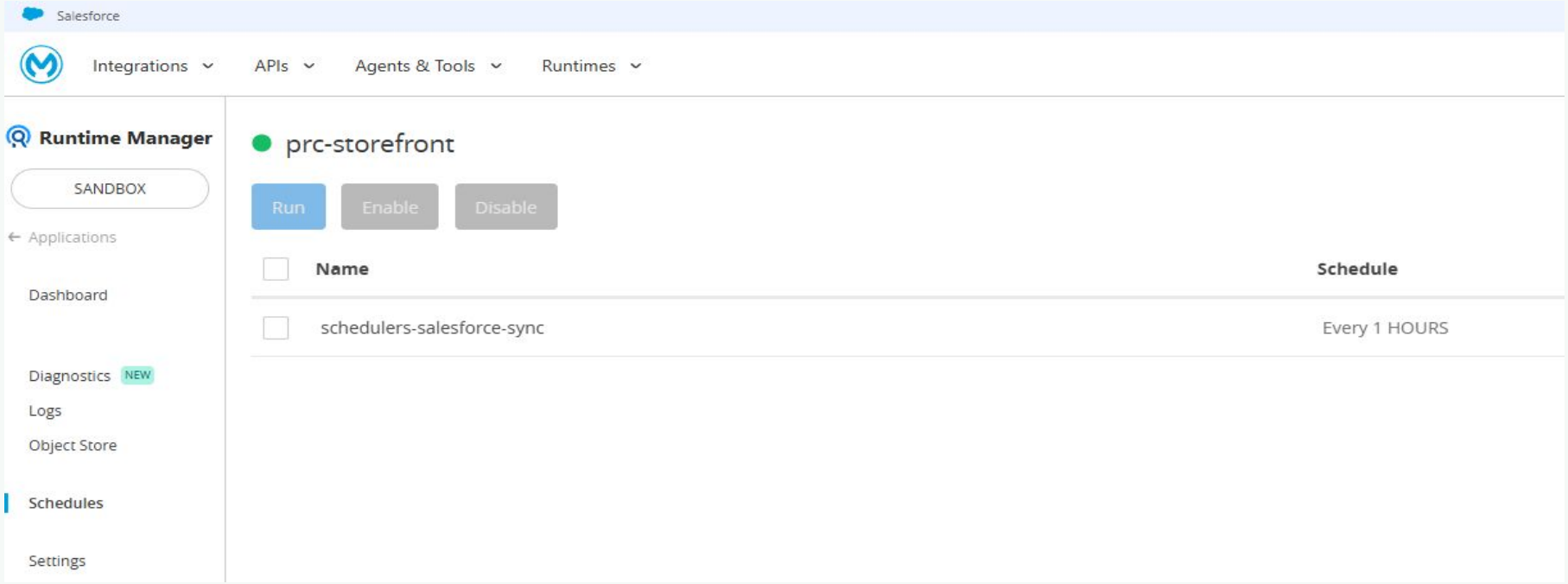
Reload

Job ID	750g5000008dsVO	Job Type	Bulk V2	Status	Job Complete
Submitted By	Mahassen Essalhi	Operation	Insert	Total Processing Time (ms)	354
Start Time	4/13/2026, 11:47 AM CET	Queued Batches	0	API Active Processing Time (ms)	258
End Time	4/13/2026, 11:47 AM CET	In Progress Batches	0	Apex Processing Time (ms)	0
Time to Complete ([hh:]mm:ss)	00:02	Completed Batches	2		
Object	Order Product	Failed Batches	0		
External ID Field		Progress	100%		
Content Type	CSV	Records Processed	10		
Concurrency Mode	Parallel	Records Failed	0		
API Version	65.0	Retries	0		

Salesforce — App Ecoform Store



Flow-02 — Launching the Scheduler from the Platform



09

Flow-03 — Order Status

GET /order-status/{orderId} · OpenWeatherMap · weatherBuffer

Flow-03 — Weather Delay Calculation Rules

+2d

Snow · Thunderstorm
Squall · Tornado

48h delay: Safety

+1d

Rain · Drizzle · Mist
Fog · Haze · Smoke

12h delay: Traffic

+0d

Clear · Clouds
(any other condition)

Standard delivery

+0d

sys-weather KO
(fallback)

On Error Continue

```
estimatedDelivery = estimated_delivery (base, 5d) + days(weatherBuffer)
```

Flow-03 — Error Handling & Fallback

WEATHER — On Error Continue

Invalid zipCode or unavailable API:

- payload = "Clear" by default
- weatherBuffer = 0
- shippingNotice = "Standard delivery"

→ The client always receives a valid response

BATCH SYNC — On Error Propagate

If the list of orders to synchronize is empty → Error message in the logs.
Each Salesforce step (Accounts, Orders, OrderItems) is wrapped in a try/on-error-propagate. A failure stops the batch and the error is propagated to the main flow. Only orders whose upsert succeeded are marked synced=true.

Postman Collections

projet solo | Search | Invite

prc-storefront PROD | GET /products | GET Retrieve order-status | GET /products

COLLECTIONS
- exp-storefront
  - products
    - GET /products
      - 200
      - 400
      - 404
      - 500
  - order-status
- prc-storefront
  - products
    - GET /products
      - 200
      - 400
      - 404
      - 500
  - order-status

ENVIRONMENTS
- exp-storefront
- exp-storefront PROD
- prc-storefront
- prc-storefront PROD
- sys-database
- sys-database PROD
- sys-salesforce
- sys-salesforce PROD

exp-storefront > products > /products

GET {{baseUrl}} /products

Docs | Params | Authorization | Headers 9 | Body | Scripts | Tests | Settings

Accept	*/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	
Accept	application/json	
client_id	{{client_id}}	
client_secret	{{client_secret}}	
Key	Value	Description

Body | Cookies | Headers 5 | Test Results — 200 OK

{} JSON | Preview | Visualize

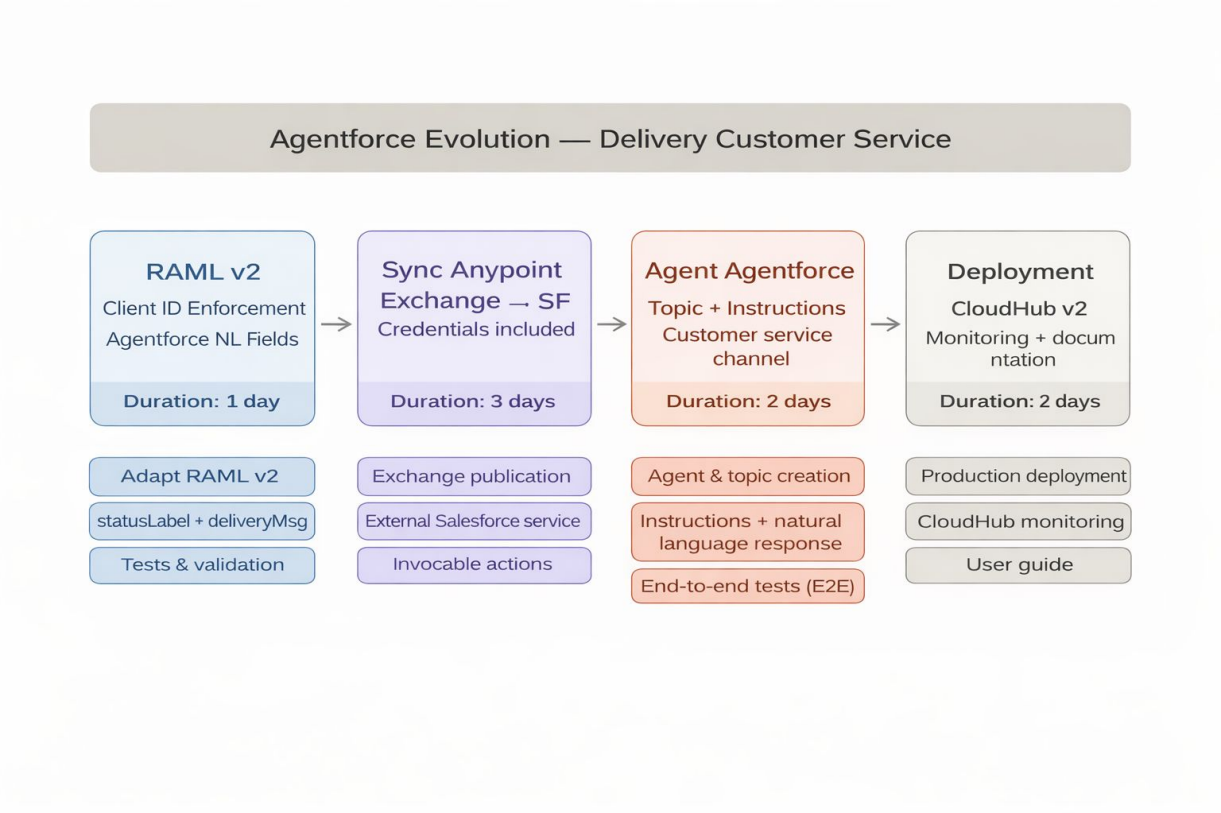

```
93          "name": "Écharpe en Cachemire Bio",
94          "category": "Accessoire",
95          "composition": "100% Cachemire Bio",
96          "price_amount": 39.9,
97          "price_currency": "EUR",
98          "color_code": "#D4A017",
99          "color_label": "Ocre Soleil",
100         "eco_score": 91,
101         "variants": "[{\"size\": \"TU\", \"color\": \"Ocre Soleil\", \"stock\": 20}]"
102     },
103     {
104         "product_id": "SHIRT-ECO-001",
105         "name": "Chemise en Coton Bio",
106         "category": "Haut",
107         "composition": "100% Coton Bio",
108         "price_amount": 49.9,
```

10

Front-End & Evolutions

Future Features

Evolutions


Agentforce Evolution — Delivery Customer Service
RAML v2
Client ID Enforcement
Agentforce NL Fields
Duration: 1 day
Sync Anypoint Exchange → SF
Credentials included
Duration: 3 days
Agent Agentforce
Topic + Instructions
Customer service channel
Duration: 2 days
Deployment
CloudHub v2
Monitoring + docum ntation
Duration: 2 days
Adapt RAML v2
statusLabel + deliveryMsg
Tests & validation
Exchange publication
External Salesforce service
Invocable actions
Agent & topic creation
Instructions + natural language response
End-to-end tests (E2E)
Production deployment
CloudHub monitoring
User guide

Planned Technical Evolutions

Weather Cache — Object Store v2 (1 day)

Cache the weatherCondition response per zipCode in Object Store v2.

Key: "weather::{zipCode}"
TTL: 30 minutes

Avoids repeated calls to OpenWeatherMap and improves performance.

Pagination GET /products (2 days)

Add pageNumber + pageSize across all 3 layers (exp/prc/sys-database).

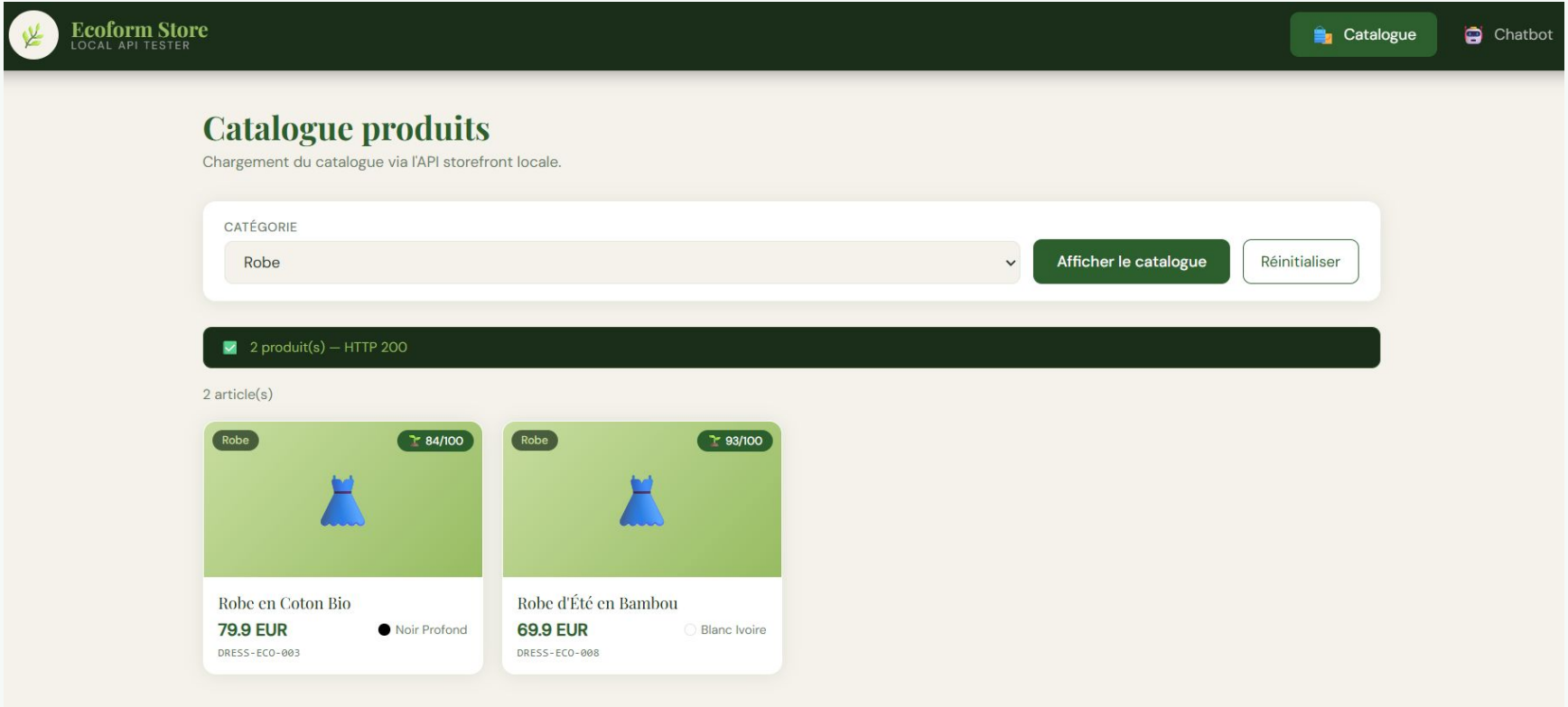
SQL cursor: LIMIT/OFFSET
totalCount in the response

E-commerce best practice for catalog scalability.

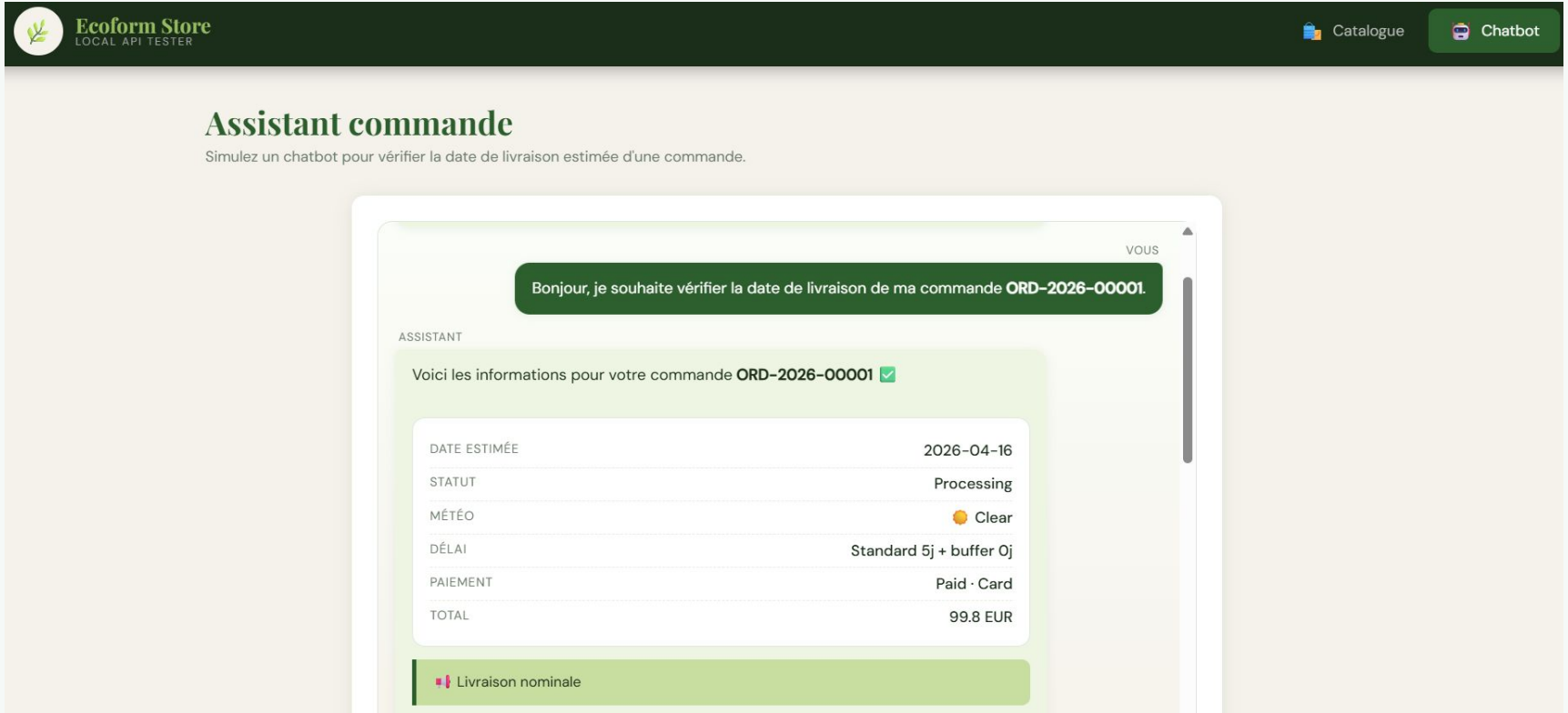
Technical Roadmap — Estimates

Object Store Cache: 1 day | Pagination: 2 days | Total: 3 estimated development days

Front-End Simulation Interface



Front-End Simulation Interface

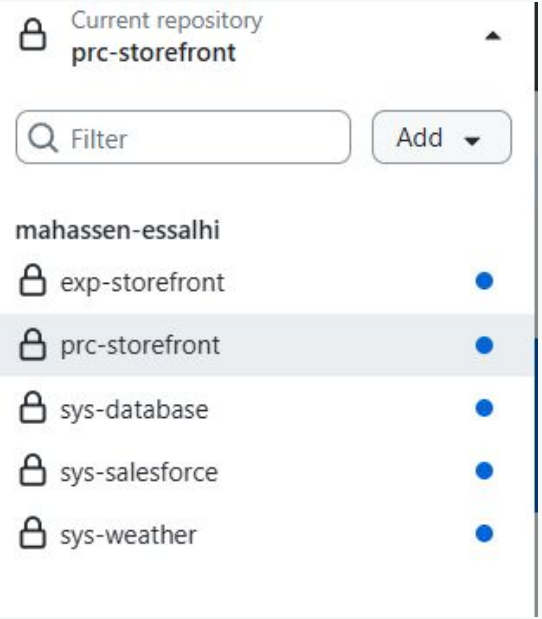


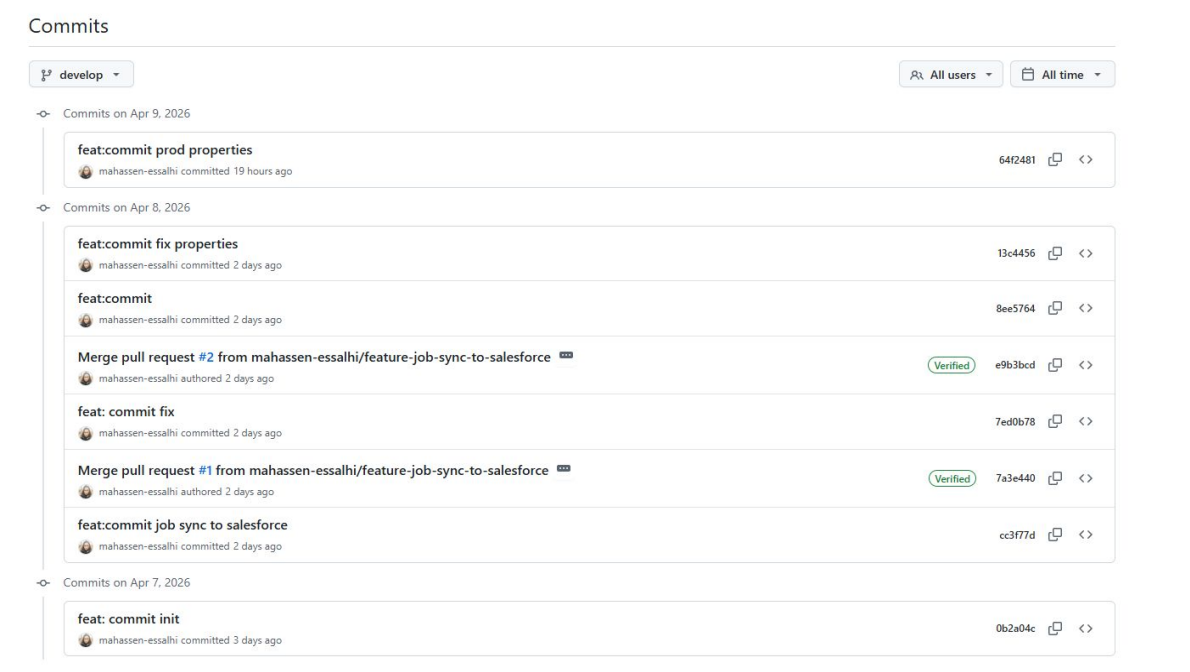
11

Git, Github — Code Versioning

Git, GitHub & Front-end simulation

Git, GitHub — Code Versioning





Demo

Ecoform Store · API-led Architecture MuleSoft / CloudHub 2.0 · Technical Presentation

Q & A

Thank you for your attention